ESENCIAL PARA EXAMEN

📌 1. Validación de Identificadores

Reglas

- **Debe comenzar por:** letra (A-Z, a-z), guión bajo `_` o signo dólar `$`-
- No contener espacios, símbolos de puntuación ni carácteres especiales excepto `_` y `$`.
- No puede ser una clave reservada de java. Ej: `class`, `for`, `static`, `null`...

📌 2. Literales Numéricos

Como identificar Sistemas de Numeración

Sistema	Prefijo	Base	Ejemplo
Decimal	*(sin prefijo)*	10	`123`
Binario	`0b` o `0B`	2	`0b1010`
Octal	`0` *(cero inicial)*	8	`0123`
Hexadecimal	`0x` o `0X`	16	`0x1A3F`

Cómo Identificar Tipo de Literal

Condición	Tipo
Lleva `.` o `e`/`E` (sin `F`)	`double`
Termina en `f` o `F`	`float`
Termina en `l` o `L`	`long`
Prefijo `0b`, `0`, `0x` (sin sufijo)	`int`
Solo dígitos (sin sufijo ni prefijo)	`int`

Reglas de Validación

Regla	Válido	No Válido
No dos puntos decimales	`1.23`	`1.34.5`
Guiones bajos (_) solo entre dígitos	`1_234`	`_234`, `234_`, `1_.23`
Exponentes solo con decimales	`1.23e5`, `3e12`	`0x1Ae5`
No `_` antes/después de `.` o `e`	`1_234.5e3`	`1234._5`, `3_e12`
Sistemas no decimales sin `.` ni `e`	`0x1A`, `0b101`	`0x1A.5`, `0b101e2`

📌 3. Literales de Tipo `char`

Reglas Clave

Regla	Válido	No Válido
Comillas simples `' '`	`'a'`	`"a"`, `a`
Solo **un** carácter	`'X'`	`'XY'`, `''`
Unicode: `\u` + 4 dígitos hex	`'\u0066'`	`'\u00G1'`
Secuencias de escape	`'\n'`, `'\t'`, `'\\'`, `'\''`	`'/n'`, `'\ñ'`
No comillas dobles	`'A'`	`"A"`

Secuencias de Escape Comunes

- `'\n'` → salto de línea
- `'\t'` → tabulación
- `'\\'` → barra invertida
- `'\''` → comilla simple
- `'\b'` → retroceso (backspace)

📌 4. Conversiones de Tipos (Casting)

Jerarquía de Tipos

```
byte → short → int → long → float → double
        char ↗
```

Conversiones entre Primitivos y Objetos

DATO ORIGEN	DATO SALIDA	MÉTODO / EJEMPLO
Primitivo	**Primitivo**	**Casting** → `(tipo) variable` 🧩 *Ejemplo:* `int i = (int) 3.14;`
Objeto (String)	**Primitivo**	**parseTipo()** → `Tipo.parseTipo("valor")` 🧩 *Ejemplo:* `int n = Integer.parseInt("42");`
Primitivo	**Objeto (Wrapper)**	**valueOf()** → `Tipo.valueOf(valor)` 🧩 *Ejemplo:* `Integer num = Integer.valueOf(25);`
Primitivo	**Cadena (String)**	**String.valueOf()** → `String.valueOf(valor)` 🧩 *Ejemplo:* `String s = String.valueOf(123);`

📌 5. Operadores Lógicos y Relacionales

Operadores Lógicos

Operador	Significado	Ejemplo

Operador	Significado	Ejemplo
`&&`	AND (Y)	`a && b`
`||`	OR (O)	`a || b`
`!`	NOT (NO)	`!a`

Orden de evaluación de operadores lógicos

```
!NOT → && → || OR → long → float → double
()↗
```

📌 6. Expresiones Algorítmicas

Operaciones con Dígitos

Operación	Fórmula	Ejemplo
Última cifra de N	`N % 10`	`123 % 10 = 3`
Quitar última cifra	`N / 10`	`123 / 10 = 12`
Primera cifra (3 dígitos)	`N / 100`	`456 / 100 = 4`
Suma de dígitos (3 cifras)	`N/100 + (N%100)/10 + N%10`	`123 → 1+2+3 = 6`
Quitar primera cifra (5 dígitos)	`N % 10000`	`12345 % 10000 = 2345`

Operaciones con Caracteres

Operación	Código ASCII	Ejemplo
Letra mayúscula	`65-90` (`'A'` a `'Z'`)	`char c = 'B'; boolean esMayus = (c >= 65 && c <= 90);`
Letra minúscula	`97-122` (`'a'` a `'z'`)	`char c = 'b'; boolean esMinus = (c >= 97 && c <= 122);`
Dígito	`48-57` (`'0'` a `'9'`)	`char c = '5'; boolean esDigito = (c >= 48 && c <= 57);`

📌 7. Orden de Prioridad de Operadores

Cuando hay múltiples operadores en una expresión, se evalúan en este orden (de mayor a menor prioridad):

Prioridad	Operadores	Descripción
1	`expr++`, `expr--`	Post-incremento, post-decremento
2	`++expr`, `--expr`	Pre-incremento, pre-decremento

Prioridad	Operadores	Descripción
3	*, /, %	Multiplicación, división, módulo
4	+, -	Suma, resta
5	<, >, <=, >=	Operadores relacionales
6	==, !=	Igualdad, desigualdad
7	&&	AND lógico
8	||	OR lógico
9	=, +=, -=	Asignación